

O que é *o MCP*

O **Model Context Protocol** é um padrão aberto para conectar aplicações de IA a sistemas externos: arquivos, bancos de dados, APIs, ferramentas e fluxos de trabalho. É o que deixa o agente do Cursor consultar o GitHub, controlar um navegador ou ler o seu banco sem você copiar e colar nada. Nesta aula: o problema que ele resolve, o que um servidor MCP oferece e como isso aparece no Cursor.

Padrão aberto

"USB-C da IA"

Servidores MCP

Tools · Resources · Prompts

Local e remoto

Segurança

Formação Claude Code 2026 · Baseado em modelcontextprotocol.io (What is MCP, Architecture, Versioning) e cursor.com (Docs: Model Context Protocol; Help: MCP integrations) consultados em outubro de 2026 (protocolo 2026-07-28, Cursor 3.22)

Cada IA com cada ferramenta: *integração demais*

Sem um padrão, cada aplicação de IA precisa de uma integração própria para cada ferramenta: o editor A fala com o GitHub de um jeito, o editor B de outro, e assim por diante. Com **N** aplicações e **M** ferramentas, são **N × M** integrações para escrever e manter.

	SEM PADRÃO	COM MCP
4 apps × 5 ferramentas	20 integrações	4 clientes + 5 servidores = 9
Nova ferramenta	Uma integração por app	Um servidor MCP, serve para todos
Novo app de IA	Uma integração por ferramenta	Fala MCP e usa todos os servidores

E PARA QUEM USA?

Sem integração, sobra o trabalho manual: copiar a issue do GitHub, colar o resultado da consulta no banco, descrever o que aparece no navegador. A documentação do Cursor resume: em vez de explicar a estrutura do projeto repetidas vezes, **integre diretamente com as suas ferramentas**.

Um padrão aberto, *como o USB-C*

"MCP (Model Context Protocol) é um padrão de código aberto para conectar aplicações de IA a sistemas externos." —
modelcontextprotocol.io

A ANALOGIA OFICIAL

Pense no MCP como uma **porta USB-C para aplicações de IA**. Assim como o USB-C padroniza a ligação entre aparelhos eletrônicos, o MCP padroniza a ligação entre aplicações de IA e sistemas externos.

Cursor

Claude

ChatGPT

VS Code

MCP

um protocolo comum

Arquivos locais

Bancos de dados

GitHub · Notion

Navegador

O MCP define só **como o contexto é trocado**. Ele não decide como a aplicação usa o modelo de IA nem o que faz com o contexto recebido.

O que o MCP *permite*

EXEMPLOS DA DOCUMENTAÇÃO OFICIAL

- Agentes que acessam seu Google Agenda e o Notion, como um assistente pessoal
- O Claude Code gerando um app web inteiro a partir de um design do Figma
- Chatbots de empresa ligados a vários bancos de dados para análise por chat
- Modelos criando designs 3D no Blender e mandando para uma impressora 3D

NO DIA A DIA COM O CURSOR

- ✓ Ler e criar issues e pull requests no **GitHub** (aula 08)
- ✓ Abrir e testar a aplicação num **navegador** (aula 09)
- ✓ Consultar o esquema e os dados de um **banco**
- ✓ Trazer requisitos do **Notion**, do **Linear** ou do **Google Drive**
- ✓ Pegar telas e medidas do **Figma**

Por que o MCP *importa*

DESENVOLVEDORES

Menos tempo e complexidade para construir uma aplicação de IA ou integrar com uma. Um servidor escrito uma vez funciona em vários clientes.

APLICAÇÕES E AGENTES

Acesso a um ecossistema de fontes de dados, ferramentas e apps, que aumenta o que eles conseguem fazer.

USUÁRIOS FINAIS

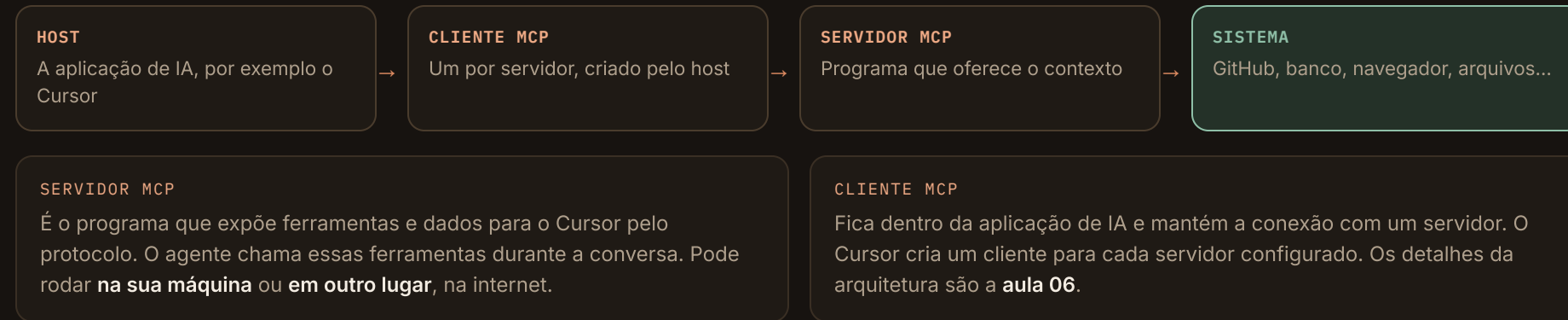
Assistentes mais capazes, que acessam seus dados e agem em seu nome quando necessário.

SUPORTE AMPLO

Assistentes como **Claude** e **ChatGPT**, ferramentas de desenvolvimento como **Visual Studio Code** e **Cursor**, e muitos outros suportam MCP. A ideia, nas palavras do site oficial: **construa uma vez e integre em todo lugar**.

Servidores MCP podem ser escritos em qualquer linguagem que consiga escrever no `stdout` ou servir um endpoint HTTP: Python, JavaScript, Go e outras. Há SDKs oficiais para C#, Go, Java, Kotlin, PHP, Python, Ruby, Rust, Swift e TypeScript.

Cliente *e servidor*



Tools, resources *e prompts*

TOOLS (FERRAMENTAS)

Funções que a IA pode **executar** para agir: operações em arquivos, chamadas de API, consultas ao banco.

RESOURCES (RECURSOS)

Fontes de dados que dão **contexto**: conteúdo de arquivos, registros do banco, respostas de API.

PROMPTS

Modelos de mensagem **reutilizáveis** que estruturam a interação com o modelo, como exemplos de uso.

EXEMPLO DA DOCUMENTAÇÃO: UM SERVIDOR DE BANCO DE DADOS

Expõe **tools** para consultar o banco, um **resource** com o esquema das tabelas e um **prompt** com exemplos de como usar as ferramentas.

Na prática, no Cursor, as **tools** são o que mais aparece: o agente decide sozinho quando chamar cada uma, do mesmo jeito que escolhe ler um arquivo ou rodar um comando. O Cursor também suporta resources, prompts e outros recursos do protocolo.

Servidores *locais e remotos*

	LOCAL	REMOTO
Onde roda	Na sua máquina, iniciado pelo Cursor	Num servidor na internet (ou na sua rede)
Transporte	<code>stdio</code> (entrada e saída padrão do processo)	<code>Streamable HTTP</code> (o Cursor também aceita <code>SSE</code>)
Como configurar	Um comando: <code>npx</code> , <code>python</code> , <code>docker</code> ...	Uma URL
Quem usa	Uma pessoa	Várias pessoas ao mesmo tempo
Autenticação	Manual, por variáveis de ambiente	OAuth ou token no cabeçalho
Exemplo	Servidor de arquivos locais, Playwright	GitHub remoto, Sentry

Para dados sensíveis, a documentação do Cursor sugere servidores **locais** com `stdio` : o servidor roda na sua máquina, sob o seu controle.

Como o MCP *aparece no Cursor*

- 1 Instalar:** pela página **Customize** (barra lateral) > **MCPs**, com um clique, ou escrevendo um `mcp.json`.
- 2 Usar:** o agente usa as ferramentas listadas em **Available Tools** quando forem relevantes, inclusive no modo Plan. Você também pode pedir pelo nome.
- 3 Aprovar:** por padrão, o Cursor pede aprovação antes de usar uma ferramenta MCP; a seta ao lado do nome mostra os argumentos. MCP segue os mesmos **Run Modes** dos comandos de terminal.
- 4 Ligar e desligar:** em **Customize**, cada servidor tem uma chave; desligado, ele não carrega nem aparece no chat.

.CURSOR/MCP.JSON • EXEMPLO DE SERVIDOR LOCAL

```
{
  "mcpServers": {
    "server-name": {
      "command": "npx",
      "args": ["-y", "mcp-server"],
      "env": { "API_KEY": "value" }
    }
  }
}
```

EXEMPLO DE SERVIDOR REMOTO

```
{
  "mcpServers": {
    "server-name": {
      "url": "http://localhost:3000/mcp",
      "headers": { "API_KEY": "value" }
    }
  }
}
```

Mais servidores *não é melhor*

Cada servidor ligado traz suas **instruções** e o **catálogo de ferramentas** para a janela de contexto (categoria "MCP" no anel de contexto, visto no módulo 35). Muitos servidores = muitas ferramentas para o modelo escolher e menos espaço para o seu código.

- ✓ Ligue só os servidores que o projeto usa
- ✓ Desligue o resto em Customize, sem remover
- ✓ Prefira servidores que deixam escolher conjuntos de ferramentas

PROJETO OU GLOBAL?

`.cursor/mcp.json` na pasta do projeto vale só ali e pode ir para o Git, para o time ter as mesmas ferramentas. `~/.cursor/mcp.json` vale em todos os projetos. Os dois são combinados; com o mesmo nome, o do projeto vence.

Se o `mcp.json` do projeto vai para o Git, **não coloque tokens nele**. Use variáveis como `${env:API_KEY}` (aula 08).

Um servidor MCP *age em seu nome*

A documentação do Cursor lembra: servidores MCP podem **acessar serviços externos e executar código** em seu nome. Entenda o que um servidor faz antes de instalar.

- ✓ **Verifique a origem:** só instale de desenvolvedores e repositórios confiáveis
- ✓ **Revise as permissões:** que dados e APIs o servidor vai acessar
- ✓ **Limite as chaves:** tokens com o mínimo de permissão necessário
- ✓ **Audite o código:** em integrações críticas, leia o código do servidor

SEGREDOS

Use variáveis de ambiente para chaves e tokens; nunca escreva o valor direto em arquivos versionados.

APROVAÇÃO

Mantenha a aprovação das ferramentas MCP ligada no começo. Libere automaticamente só as ferramentas de **leitura** em que você confia.

INJEÇÃO DE PROMPT

Conteúdo vindo de fora (uma issue, uma página web) pode tentar dar ordens ao agente. Desconfie de ações que você não pediu.

Teste o que *ficou*

1. O que é o MCP?

- A) Um modelo de IA do Cursor
- B) Um padrão aberto para conectar apps de IA a sistemas externos
- C) Uma extensão do VS Code
- D) Um banco de dados

B. O "USB-C das aplicações de IA".

2. Quais são as três primitivas que um servidor MCP oferece?

- A) Tools, resources e prompts
- B) Rules, skills e hooks
- C) Agent, Ask e Plan
- D) GET, POST e DELETE

A. Ações, dados de contexto e modelos de mensagem.

3. Um servidor configurado com um comando `npx` é...

- A) Remoto, via HTTP
- B) Local, via stdio
- C) Um plugin de navegador
- D) Uma rule

B. Remoto se configura com URL.

4. Por que não ligar todos os servidores MCP de uma vez?

- A) O Cursor só aceita um
- B) Cada servidor ocupa contexto e amplia o que pode dar errado
- C) Eles apagam as rules
- D) Não há motivo

B. Ligue só o que o projeto usa.

Um protocolo, *muitas ferramentas*

O MCP troca $N \times M$ integrações por um padrão comum. **Servidores** oferecem **tools**, **resources** e **prompts**; rodam **localmente** (stdio) ou **remotamente** (HTTP). No Cursor, instale pelo **Customize** ou pelo `mcp.json`, ligue só o que usa e trate cada servidor como alguém agindo em seu nome.

Padrão aberto

Host · cliente · servidor

stdio · Streamable HTTP

Customize › MCPs

Menos é mais

Próxima aula → 06 · Arquitetura do Protocolo MCP

Fontes: modelcontextprotocol.io (What is MCP, Architecture overview, Versioning); github.com/modelcontextprotocol/servers (lista de SDKs); cursor.com/docs/mcp; cursor.com/help/customization/mcp, consultados em outubro de 2026.